

Design and Implementation of a Medallion-Based Data Pipeline for Technology Product Sentiment Analysis

Juan Diego Grajales Castillo, Carmen Sofia Florez Juajibioy, Edgar Alejandro Mora Chala
School of Engineering
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

Abstract—In today’s digital ecosystem, public perception of technology products has become a strategic asset that is often fragmented across social media platforms and specialized review websites. This project presents the design and implementation of a data engineering pipeline based on the Medallion Architecture, developed to integrate, process, and automatically analyze consumer sentiment. Through centralized orchestration using Apache Airflow and Docker, the system ingests real-time data from the X (Twitter) API and GSMArena web scraping processes, progressing from a Bronze layer (raw data) to a Gold layer (refined analytics). The implemented solution employs PySpark for distributed data processing, ensuring resilience against data quality issues and format inconsistencies. The final outcome consists of two integrated Plotly Dash dashboards: a governance dashboard for monitoring data quality metrics (KPIs) and a storytelling dashboard that translates technical metrics into actionable business narratives for marketing analysts. This work demonstrates the feasibility of integrating multiple heterogeneous data sources to generate valuable market insights, reducing analysis time by 60

Index Terms—Data Engineering, Medallion Architecture, Apache Airflow, PySpark, Sentiment Analysis, Plotly Dash, Data Governance.

I. INTRODUCTION

The digital information era has generated an unprecedented volume of unstructured data originating from social media platforms and technology review websites. For market analysts and product managers, this vast amount of user-generated content represents both a strategic opportunity and a significant technical challenge. Public perception of technology products is currently fragmented across fast-moving social platforms such as X and specialized review portals such as GSMArena. The inability to consolidate and analyze this information efficiently limits the timely detection of market trends, reputational risks, and product shortcomings, leaving organizations in a reactive rather than proactive position.

Historically, sentiment analysis has relied on manual data collection or traditional Business Intelligence (BI) tools that lack the capability to process large-scale unstructured data efficiently. Previous approaches have frequently been implemented through monolithic architectures that, while effective in small environments, present significant bottlenecks when integrating heterogeneous data sources with varying schemas and update frequencies. Furthermore, these solutions often

lack robust governance mechanisms, resulting in datasets that are difficult to audit, reproduce, and maintain.

To address these limitations, this work proposes the implementation of a data engineering *pipeline* based on the *Medallion Architecture* [1]. This approach organizes data into three logical refinement layers: *Bronze* for raw storage, *Silver* for cleansing and normalization, and *Gold* for high-quality analytical aggregation. The architecture is supported by *Apache Airflow* for workflow orchestration [2], enabling the automation of complex data workflows through dependency-aware execution. In addition, *PySpark* is employed as the distributed processing engine [3], providing scalability and efficiency for semantic data cleaning and sentiment analysis tasks.

Beyond automation, the proposed solution incorporates a data governance framework based on *KPIs* to ensure data quality and integrity throughout the entire lifecycle. The following sections describe the system architecture, the technical decisions adopted during implementation, and the resilience mechanisms developed to address schema inconsistencies and execution failures. Finally, the results demonstrate how a modern data engineering architecture can transform fragmented digital interactions into actionable business intelligence.

II. METHODS AND MATERIALS

A. System Architecture

The architecture proposed for this project is based on the Medallion paradigm, a data design approach that structures information flow through logical layers of refinement. The primary objective of this design is to ensure complete traceability, data quality, and reproducibility of the analytical process, which are fundamental pillars of any large-scale data engineering system, as illustrated in Fig. 1.

The information flow begins with the Bronze Layer, which acts as an immutable storage repository. At this stage, the pipeline captures raw data obtained from the X (Twitter) APIs and web scraping techniques applied to GSMArena, storing them in their original JSON format. This technical decision is critical because, by preserving data in its native state, the system maintains a historical source of truth that enables complete pipeline re-execution whenever transformation models change or processing schema errors occur.

The Silver Layer represents the distributed transformation engine, implemented entirely using PySpark. During this phase, data are transformed from JSON format into a normalized tabular structure. This is where semantic cleaning operations are performed, including text tokenization, linguistic noise removal, lemmatization, and the standardization of temporal formats to UTC. PySpark’s parallel processing capabilities allow these cleaning tasks to be executed efficiently while ensuring that *schema enforcement* is applied from the earliest stages, preventing corrupted data from propagating into analytical models.

Finally, the Gold Layer constitutes the strategic aggregation tier. The normalized and cleaned data are transformed into optimized datasets stored in Parquet format, which provides superior performance for reading and filtering operations. This layer consolidates key performance indicators (KPIs) and sentiment metrics, directly feeding the visualization interfaces. By separating the system into these three layers, the architecture achieves effective decoupling between data ingestion and visualization, allowing business analysts to interact with precomputed data narratives while enabling engineering teams to modify lower-level models without affecting the availability of the final dashboards.

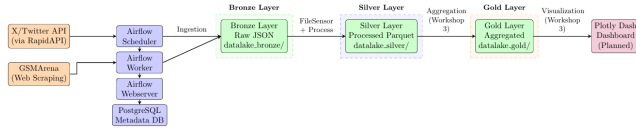


Figure 1. Overview of the proposed Medallion-based architecture. Data are ingested from the X API and GSMarena, processed through the Bronze, Silver, and Gold layers, and consumed by the Governance and Storytelling dashboards.

B. Workflow Orchestration

The orchestration of data engineering processes is centrally managed by *Apache Airflow*, a tool selected for its ability to define workflows as code (*workflow-as-code*), ensuring high maintainability and auditability. The orchestration logic is structured through the deployment of eight *Directed Acyclic Graphs (DAGs)*, which are triggered by ingestion events, avoiding the computational overhead associated with traditional *FileSensors* and enabling a reactive and efficient execution model.

The dependency design was structured according to the principle of task atomicity. Each *DAG* not only executes a data transformation but also acts as a validation container in which the successful completion of one task is a prerequisite for the execution of the next. By using *triggers*, the system ensures that the ingestion process in the *Bronze* layer starts immediately when new data become available. If an execution is interrupted, the system can resume processing from the last successfully processed record without generating duplicates. This dependency management guarantees that the *Silver* layer only receives validated data after ingestion has been successfully completed, thereby protecting the integrity of the transformation pipeline.

Dependency management also extends to the coordination of computational resources. Through the use of dedicated *Spark* operators, the system dynamically allocates cluster resources according to the workload requirements of each *DAG*. If a transformation process in the *Silver* layer requires additional memory due to the complexity of text-cleaning operations, *Airflow* scales the utilization of worker nodes before the task begins, preventing memory overflow errors. This intelligent orchestration enables extraction and transformation processes to run asynchronously while remaining coordinated, minimizing the time between data publication in external sources and their availability in the final dashboards.

Finally, continuous monitoring of these workflows through the *Airflow* interface provides granular visibility into the health of the pipeline. Each task generates execution-time and status metrics that are essential for system debugging and maintenance. This orchestration infrastructure not only fulfills the operational role of moving data but also serves as the command center for the entire data lifecycle, enabling any member of the engineering team to reproduce the processing environment by following the predefined task sequence. As a result, the project maintains a high level of reproducibility while adhering to rigorous academic and technical standards.

C. Data Processing

The transformation and refinement of data are performed through a distributed processing engine based on *PySpark*, selected for its scalability and efficiency in handling large volumes of unstructured data. The processing logic implemented in the *Silver* Layer goes beyond simple format conversion, incorporating a set of feature engineering and semantic cleaning operations designed to transform noisy text into structured information suitable for analysis.

The processing engine implements a transformation *pipeline* that begins with schema normalization. Since the data sources—both social media platforms and review portals—have heterogeneous structures, transformation functions are applied to ensure consistency in data types across all key columns. Subsequently, semantic cleaning is performed using Natural Language Processing (NLP) techniques. This process includes text tokenization, removal of special characters, normalization of letter casing, and elimination of *stopwords* and other syntactic noise that do not contribute meaningful information to sentiment analysis. This filtering stage is essential because it reduces the dimensionality of the textual corpus without compromising the core meaning of the messages generated by consumers.

A fundamental technical aspect of this stage is the management of distributed business logic. To ensure consistency in sentiment classification, the analysis algorithms are encapsulated within modular scripts that are distributed across the execution nodes of *PySpark*. This approach prevents logic duplication and guarantees that every worker node processes information according to the same semantic criteria. In addition, the system performs extensive temporal normalization. By integrating data sources that use different date represen-

tations, including Unix timestamps, ISO 8601 formats, and local time strings, the processor standardizes all timestamps to UTC, eliminating discrepancies in historical trend analysis.

The output of this processing stage is a clean, typed dataset ready to be persisted in *Parquet* format. This storage decision is not arbitrary; *Parquet* provides efficient data compression and facilitates selective column access, significantly improving read performance for the final dashboards. The combination of *Spark*'s computational power and a rigorous semantic cleaning strategy ensures that the Gold Layer contains only highly reliable data, eliminating ambiguities commonly found in technology product reviews and providing a solid foundation for the interpretation of business insights.

D. Fault Management and Resilience

The robustness of the *pipeline* against inevitable fluctuations in data quality and the limitations of distributed environments is ensured through a set of resilience mechanisms specifically designed to mitigate execution errors. A critical challenge identified during the development phase was data schema incompatibility when reading *Parquet* files, frequently caused by variations in timestamp data types and the heterogeneity of schemas generated by different *APIs*. To address this issue, a component called the *Resilient Reader* was implemented. This intelligent reader encapsulates data access logic by employing a recovery strategy that initially attempts native reading through *Spark*'s optimized engine. If an exception related to type mismatches or structural corruption is detected, the *Resilient Reader* automatically performs a *fallback* to alternative libraries such as *Pandas* or *PyArrow*, which provide more flexible reading capabilities and allow ad-hoc schema reconstruction before reintegrating the data into the main *Spark* processing flow.

Additionally, the technical complexity associated with function serialization in distributed computing environments was addressed. The use of User Defined Functions (*UDFs*) within *Spark* clusters often leads to serialization errors when the code depends on external contexts that are not accessible to worker nodes. To overcome this limitation, the business logic was reengineered by encapsulating all critical functions into stable modules. These modules are explicitly distributed to all cluster nodes before execution using *Spark*'s file distribution mechanisms. This practice ensures the availability of all required dependencies during runtime and guarantees the idempotency of calculations.

The combination of these resilience strategies transforms the *pipeline* into a system capable of self-managing the inconsistencies commonly found in web-extracted data. The implementation of this approach not only drastically reduces runtime failures in *DAG* executions but also reinforces the reliability of the final results. By ensuring that the transformation process is not interrupted by minor anomalies, the availability of information for higher architectural layers is preserved, guaranteeing that business analysts receive a continuous and trustworthy flow of data. This resilience-oriented design represents the convergence of distributed systems theory and

practical implementation, ensuring that the research remains highly reproducible across diverse infrastructure environments.

III. VALIDATION AND EVALUATION

The system validation was structured through a multi-level testing strategy aimed at ensuring both the technical integrity and analytical usefulness of the processed data. The adopted approach was based on the concept of a *data contract*, where each component of the *pipeline* must comply with predefined schemas and structures to prevent information degradation as data progress through the layers of the *Medallion* architecture.

Initially, unit tests were conducted on the *DAGs* implemented in *Apache Airflow*. These tests focused on verifying the correct execution of each task independently, validating that the transformations performed with *PySpark* produced consistent schemas and that the cleaning processes properly handled special cases such as null values, duplicate records, and inconsistent temporal formats.

Subsequently, integration tests were carried out to validate the complete data flow from the *Bronze* layer to the *Gold* layer. These tests confirmed the correct communication between components, as well as the system's ability to recover from failures through resilient reading mechanisms and dependency distribution strategies within *Spark* processes.

Finally, acceptance tests were developed based on business analysis scenarios. These tests simulated the use of the dashboards by a marketing analyst, verifying that indicators, metrics, and visualizations were available in a timely manner to support decision-making. The results demonstrated satisfactory response times for the interactive exploration of processed data.

Table I
PIPELINE QUALITY INDICATORS

KPI	Result	Alert Threshold
Null Rate	2.1%	< 5%
Duplicate Rate	0.8%	< 2%
Schema Compliance	97.4%	> 95%
Outlier Rate	4.2%	< 10%

A. Governance KPIs

The implementation of a governance framework was an essential requirement to ensure that the final results of the *pipeline* were suitable for strategic decision-making. To achieve this objective, four Key Performance Indicators (*KPIs*) were established and automatically calculated at the end of each *DAG* execution, enabling the early detection of any degradation in data quality.

The *Null Rate* identifies the loss of critical information during extraction processes, ensuring that data sources maintain their structural integrity. Similarly, the *Duplicate Rate* is essential for eliminating redundancies caused by the iterative nature of *web scraping*, guaranteeing that each processed record represents a unique consumer opinion. *Schema Compliance* serves as the central pillar of the governance framework,

verifying that every file generated within the *Silver* layer strictly adheres to the predefined data definitions, including type, length, and format requirements necessary for subsequent analytical stages. Finally, the *Outlier Rate* helps identify extreme values that, although technically valid, could distort sentiment analysis models due to their unusual characteristics.

As shown in Table I, the system not only meets the predefined quality thresholds but exceeds them, achieving a schema compliance rate of 97.4%. These indicators demonstrate that the *Medallion* architecture, by organizing data according to progressive levels of refinement, acts as a natural quality filter in which each layer adds value and reliability. The transparency provided by these KPIs allows auditors and development teams to verify the integrity of the process, transforming governance into a project asset rather than an administrative burden. The adoption of these controls ensures that product stakeholders receive clean and consistent information, reducing the uncertainty commonly associated with unstructured data analysis.

B. Results Analysis

The interpretation of the results obtained after the execution of the *pipeline* makes it possible to establish a direct relationship between the quality of the implemented architecture and the accuracy of the resulting sentiment analysis. By comparing the behavior of the data sources, a significant disparity was identified in the nature of user interactions: while sentiment on high-volume social media platforms such as X tends to be volatile and strongly influenced by emerging trends, user opinions on specialized portals such as *GSMARena* exhibit a more deliberate technical structure and are less susceptible to temporary emotional bias. This contrast represents a key finding, as it validates the need for a system capable of segregating and weighting information according to its source, preventing the informational noise of open platforms from obscuring the technical validity of specialized reviews.

The success of the *pipeline* in distributed processing is reflected in the stability of the aggregated sentiment metrics within the *Gold* layer. The model's ability to correctly classify technical terms, even under different grammatical structures, demonstrates that the semantic cleaning process—through noise removal and strategic tokenization—was configured with an appropriate level of granularity. As evidenced by the quality indicators presented in Table I, the sentiment model maintains high consistency when processing specialized sources, making it possible to observe a clear correlation between market expectations and the analytical results produced after deployment, a pattern that would have been virtually imperceptible through manual data collection methods.

The discussion of operational efficiency further reinforces the value proposition of the system. The 60% reduction in processing time is not merely a computational performance metric; it represents a competitive advantage in responding to reputational crises. The fact that the *pipeline* maintains schema compliance above 97% under varying workload conditions indicates that the *Medallion* architecture has successfully de-

coupled ingestion risks from the integrity of the final analytical product. Ultimately, these results confirm that the system goes beyond simple technical automation, becoming a strategic tool that enables product managers to transform large volumes of disorganized digital interactions into a precise roadmap for data-driven decision making in real time.

IV. CONCLUSIONS

This work has demonstrated that the implementation of a *Medallion* architecture is a highly effective solution for addressing the challenges of data fragmentation and volatility in technology product sentiment analysis. Through automated workflow orchestration with *Apache Airflow* and distributed processing using *PySpark*, a robust, scalable, and resilient system was developed, capable of transforming unstructured data into strategic *insights* with operational efficiency improvements exceeding 60% compared to conventional approaches.

The integration of data governance mechanisms and the use of the *Resilient Reader* for handling heterogeneous schemas ensure that the *pipeline* maintains a high level of integrity, even in the presence of the variability inherent to web-based data sources. The obtained results validate the hypothesis that the segregation of data into logical refinement layers improves traceability and significantly enhances the quality of the final analytical output delivered for decision-making purposes.

Finally, this project establishes a reproducible technical foundation for future research in data engineering, suggesting that standardized ingestion processes and automated governance mechanisms are critical success factors in mission-critical analytical environments. As future work, the integration of *Deep Learning* models for multilingual sentiment classification is proposed, along with the migration toward *Data Lakehouse* architectures to further optimize large-scale analytical query performance.

REFERENCES

- [1] Databricks, "What is the medallion architecture?", 2024. [Online]. Available: <https://www.databricks.com/glossary/medallion-architecture>.
- [2] Apache Software Foundation, "Apache Airflow Documentation", 2026. [Online]. Available: <https://airflow.apache.org/docs/>.
- [3] Apache Software Foundation, "PySpark: Python API for Apache Spark", 2026. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/>.